

Resolução do Problema da Mochila Multidimensional Aplicada à Alocação de Recursos em Nuvem com Simulated Annealing

Gabriel Kruger Brum¹, João Pedro Rocha da Silva¹

¹Centro de Desenvolvimento Tecnológico
Universidade Federal de Pelotas (UFPel) – Pelotas, RS – Brasil

{gkbrum, jpdrsilva}@inf.ufpel.edu.br

Resumo. Este trabalho aborda o Problema da Mochila Multidimensional (Multidimensional Knapsack Problem – MKP), um problema clássico de otimização combinatória NP-difícil, aplicado ao contexto de alocação de máquinas virtuais em ambientes de computação em nuvem. A solução proposta utiliza a meta-heurística Simulated Annealing combinada com uma heurística de reparação gulosa para tratar as restrições de capacidade. O resolvedor foi implementado em C++ e avaliado com instâncias de teste da OR-Library, alcançando qualidade média de 99,71% em relação aos valores ótimos conhecidos.

1. Introdução

A alocação eficiente de recursos computacionais é um dos desafios centrais da computação em nuvem. Provedores de serviços como AWS, Azure e Google Cloud precisam decidir quais máquinas virtuais (VMs) alocar em um servidor físico de modo a maximizar o faturamento, sem violar os limites de CPU, memória RAM, armazenamento e largura de banda da rede [7].

Esse cenário pode ser modelado como o **Problema da Mochila Multidimensional** (Multidimensional Knapsack Problem – MKP). A ideia é simples: dado um conjunto de itens (VMs), cada um com um lucro e um consumo em várias dimensões de recurso, o objetivo é escolher quais itens incluir para obter o maior lucro possível, sem ultrapassar a capacidade de nenhum recurso. Em termos formais:

$$\max \sum_{j=1}^n p_j x_j \quad \text{s.a.} \quad \sum_{j=1}^n r_{ij} x_j \leq c_i, \quad i = 1, \dots, m, \quad x_j \in \{0, 1\} \quad (1)$$

onde p_j é o lucro do item j , r_{ij} é quanto o item j consome do recurso i , e c_i é a capacidade máxima do recurso i .

Como o MKP é um problema NP-difícil, não se conhece algoritmo que o resolva de forma exata em tempo viável para instâncias grandes. Por isso, neste trabalho utiliza-se a meta-heurística **Simulated Annealing** (SA) [1], uma técnica probabilística inspirada no processo de recozimento de metais na metalurgia.

2. Intratabilidade do Problema

O Problema da Mochila 0/1, com uma única restrição de capacidade, é reconhecidamente NP-completo [2]. A prova clássica utiliza uma redução a partir do problema Subset-Sum: dado um conjunto de números inteiros e um valor-alvo, deseja-se saber se existe algum subconjunto cuja soma é exatamente igual ao alvo. Esse problema pode ser diretamente transformado em uma instância do Knapsack.

O MKP generaliza o Knapsack ao adicionar múltiplas restrições simultâneas. Como o Knapsack comum (com apenas uma restrição) já é um caso particular do MKP, resolver o MKP não pode ser mais fácil do que resolver o Knapsack. Portanto, o **MKP também é NP-difícil** [3].

Na prática, isso significa que instâncias com centenas de itens e dezenas de restrições não podem ser resolvidas de forma exata em tempo razoável, o que justifica o uso de meta-heurísticas como o Simulated Annealing.

3. Solução Proposta

3.1. Simulated Annealing

O Simulated Annealing (SA) é uma técnica de busca que funciona de forma análoga ao processo de resfriamento de metais: no início, com a “temperatura” alta, o algoritmo aceita soluções piores com mais facilidade, o que permite explorar amplamente o espaço de busca. Conforme a temperatura diminui gradualmente, o algoritmo se torna mais seletivo e passa a aceitar apenas melhorias, convergindo para uma boa solução [1].

O controle de aceitação segue o **Critério de Metropolis**: se a nova solução é melhor, ela é aceita imediatamente; se é pior, ainda pode ser aceita com uma probabilidade que depende de quão pior ela é e da temperatura atual. Essa característica permite que o SA escape de ótimos locais, ao contrário de métodos de busca puramente gulosos.

A temperatura é reduzida multiplicando-a por um fator α (entre 0 e 1) a cada rodada, e o processo continua até atingir uma temperatura mínima. Em cada patamar de temperatura, são realizadas L tentativas de melhoria.

3.2. Geração de Vizinhos

Para gerar uma nova solução candidata a partir da solução atual, o algoritmo escolhe aleatoriamente um item e inverte sua seleção: se ele estava incluído, é removido; se não estava, é adicionado. Essa operação simples (*bit-flip*) pode gerar soluções que violam alguma restrição de capacidade, sendo necessário corrigi-las.

3.3. Heurística de Reparação Gulosa

Quando uma solução gerada viola alguma restrição, é preciso “consertá-la” removendo itens até que todas as capacidades sejam respeitadas. A questão é: quais itens remover?

A estratégia adotada é calcular, para cada item selecionado, uma razão entre o seu lucro e o quanto ele consome de cada recurso (proporcionalmente à capacidade). Itens com a pior razão – ou seja, que consomem muito recurso para pouco lucro – são removidos primeiro. Essa abordagem gulosa preserva os itens mais “valiosos” e descarta os menos eficientes, restaurando a viabilidade da solução com o menor prejuízo possível ao lucro total.

3.4. Pseudocódigo

O Algoritmo 1 resume o procedimento completo.

Algorithm 1 Simulated Annealing para o MKP

Require: Instância MKP, temperatura inicial, fator de resfriamento, temperatura mínima, iterações por patamar

Ensure: Melhor solução encontrada

```
1: Iniciar com solução vazia (reparada)
2: Guardar como melhor solução global
3: Definir temperatura inicial
4: while temperatura > temperatura mínima do
5:   for cada iteração do patamar do
6:     Gerar vizinho (inverter um item aleatório)
7:     Reparar vizinho se necessário (heurística gulosa)
8:     if vizinho é melhor then
9:       Aceitar vizinho como solução atual
10:      if melhor que o recorde global then
11:        Atualizar recorde global
12:      end if
13:    else
14:      Aceitar vizinho com probabilidade baseada na temperatura
15:    end if
16:  end for
17:  Reduzir temperatura
18: end while
19: return melhor solução encontrada
```

4. Implementação

O resolvidor foi implementado em **C++17** e organizado em módulos com responsabilidades bem definidas:

- `instance.h/.cpp` – Leitura e armazenamento dos dados de entrada no formato OR-Library;
- `solution.h/.cpp` – Representação da solução, verificação de viabilidade, reparação gulosa e geração de vizinhos;
- `sa.h/.cpp` – Implementação do Simulated Annealing;
- `main.cpp` – Interface interativa por linha de comando, com opção de execução individual ou em lote (*batch*).

A compilação é feita por um Makefile com três opções: `make` (compilação otimizada), `make static` (binário portátil com linkagem estática) e `make clean` (limpeza).

O modo em lote (*batch*) executa todas as instâncias de um arquivo de teste automaticamente e gera uma tabela com o lucro encontrado, a qualidade em relação ao ótimo conhecido, o tempo de execução e se a solução é viável.

5. Resultados Experimentais

O resolvidor foi testado com instâncias clássicas da OR-Library [4]: o arquivo `mknapi.txt` (7 instâncias de pequeno a médio porte) e o arquivo `mknapi.cb1.txt` (30 instâncias de Chu & Beasley [5] com 100 itens e 5 restrições cada). Os parâmetros utilizados foram: temperatura inicial = 1000, fator de resfriamento = 0,98, temperatura mínima = 0,01 e 1000 iterações por patamar.

5.1. Resultados do `mknapi.txt`

A Tabela 1 mostra os resultados para as 7 instâncias, que variam de 6 a 50 itens.

Tabela 1. Resultados para as instâncias `mknapi.txt`

Inst.	Itens	Restr.	Ótimo	SA	Qualidade	Tempo (ms)
1	6	10	3.800	3.800	100,00%	234,8
2	10	10	8.706,1	8.706,1	100,00%	289,6
3	15	10	4.015	4.015	100,00%	322,6
4	20	10	6.120	6.120	100,00%	406,4
5	28	10	12.400	12.400	100,00%	447,2
6	39	5	10.618	10.540	99,27%	563,9
7	50	5	16.537	16.317	98,67%	682,6
Qualidade Média					99,71%	

Nas cinco primeiras instâncias (até 28 itens), o algoritmo encontrou a **solução ótima exata** em todas elas. Para problemas desse tamanho, o SA consegue explorar o espaço de busca de forma completa o suficiente para convergir ao melhor resultado possível.

Nas instâncias 6 e 7, com 39 e 50 itens respectivamente, a qualidade ficou acima de 98,6%. Essa pequena queda é natural: com mais itens, o número de combinações possíveis cresce muito, e o algoritmo não consegue testar todas. Mesmo assim, a diferença máxima para o ótimo foi de apenas 1,33%, mostrando que a heurística de reparação preserva bem os itens mais valiosos.

O tempo de execução variou de 235 ms a 683 ms, todos abaixo de 1 segundo, o que é adequado para aplicações práticas.

5.2. Resultados do `mknapi.cb1.txt`

Para testar com instâncias maiores, foram executadas as 30 instâncias de Chu & Beasley [5], cada uma com 100 itens e 5 restrições. A Tabela 2 agrupa os resultados em três faixas. Todas as 30 soluções foram **viáveis**.

Os lucros médios variam entre as faixas porque as instâncias possuem configurações diferentes de restrições. O tempo médio foi de cerca de 792 ms por instância, com o maior tempo sendo 1.210 ms. Instâncias com restrições mais apertadas tendem a demorar um pouco mais, pois o algoritmo precisa reparar mais soluções durante a busca.

O resultado mais importante é que **todas as 30 soluções foram viáveis**, ou seja, nenhuma violou as restrições de capacidade. Isso confirma que o mecanismo de reparação funciona corretamente mesmo em instâncias maiores.

Tabela 2. Resultados agregados para as 30 instâncias `mknapcb1.txt` (100 itens, 5 restrições)

Faixa	Qtd.	Lucro Médio	Tempo Médio (ms)	Viáveis
1–10	10	23.956	701,4	10/10
11–20	10	43.136	896,9	10/10
21–30	10	60.265	774,6	10/10
Total		42.452	791,7	30/30

5.3. Síntese dos Resultados

A partir dos dois conjuntos de teste (37 instâncias no total), pode-se destacar:

- **Qualidade:** média de 99,71% no conjunto com ótimos conhecidos, com 5 de 7 instâncias resolvidas de forma ótima.
- **Tempo:** todas as instâncias foram resolvidas em menos de 1,3 segundo, mesmo com 100 itens.
- **Viabilidade:** 100% das soluções respeitaram todas as restrições de capacidade.

6. Conclusão

Este trabalho apresentou uma solução para o Problema da Mochila Multidimensional usando Simulated Annealing com heurística de reparação gulosa, no contexto de alocação de máquinas virtuais em servidores de nuvem.

Os testes com 37 instâncias da OR-Library mostraram que a solução encontra resultados de alta qualidade (99,71% de média) em menos de 1 segundo, com todas as soluções respeitando as restrições. Esses resultados confirmam que técnicas aproximativas são uma alternativa eficaz para problemas NP-difíceis, tema central da disciplina de Algoritmos e Estruturas de Dados III.

A principal lição prática é que o Simulated Annealing, quando combinado com uma boa estratégia de reparação, consegue lidar bem com problemas que possuem várias restrições simultâneas – algo comum em cenários reais como a alocação de recursos computacionais.

Como possíveis melhorias futuras, sugere-se: ajustar os parâmetros do algoritmo de forma automática durante a execução, combinar o SA com outras técnicas (como algoritmos genéticos), e testar com instâncias ainda maiores para avaliar os limites da abordagem.

Referências

- [1] Kirkpatrick, S., Gelatt, C. D. e Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, 220(4598):671–680.
- [2] Garey, M. R. e Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- [3] Kellerer, H., Pferschy, U. e Pisinger, D. (2004). *Knapsack Problems*. Springer-Verlag, Berlin.
- [4] Beasley, J. E. (1990). OR-Library: Distributing Test Problems by Electronic Mail. *Journal of the Operational Research Society*, 41(11):1069–1072.

- [5] Chu, P. C. e Beasley, J. E. (1998). A Genetic Algorithm for the Multidimensional Knapsack Problem. *Journal of Heuristics*, 4(1):63–86.
- [6] Drexl, A. (1988). A simulated annealing approach to the multiconstraint zero-one knapsack problem. *Computing*, 40(1):1–8.
- [7] Patel, Y. S., Mehrotra, N. e Soner, S. (2015). Green Cloud Computing: A Review on Energy Efficient Resource Management. *International Journal of Computer Applications*, 131(14):38–42.